

PostgreSQL 18 HA with Patroni and PgBouncer on Kubernetes

Zalando Postgres Operator • 3 database instances • Patroni HA • PgBouncer pooler

Item	Lab configuration
Platform	Kubernetes with kubectl
PostgreSQL	18
HA manager	Patroni, managed through Zalando Postgres Operator
Database instances	3
Connection pooler	PgBouncer, 2 instances
Operator release used in commands	v2.0.2
Cluster CR name	acid-postgres-ha
Namespace	postgres-ha
Pooler service	acid-postgres-ha-pooler
Estimated duration	3–4 hours

1. Why This Manual Replaces the Earlier Raw StatefulSet Lab

The earlier training manifest launched the Spilo image directly in a hand-written StatefulSet. In the live test, all three pods became independent Patroni/PostgreSQL primaries with scope 'dummy' and different PostgreSQL database system identifiers. That is not a three-member HA cluster.

Observed pattern from the failed lab:

```
postgres-0 -> scope=dummy -> role=primary -> system ID A
postgres-1 -> scope=dummy -> role=primary -> system ID B
postgres-2 -> scope=dummy -> role=primary -> system ID C
```

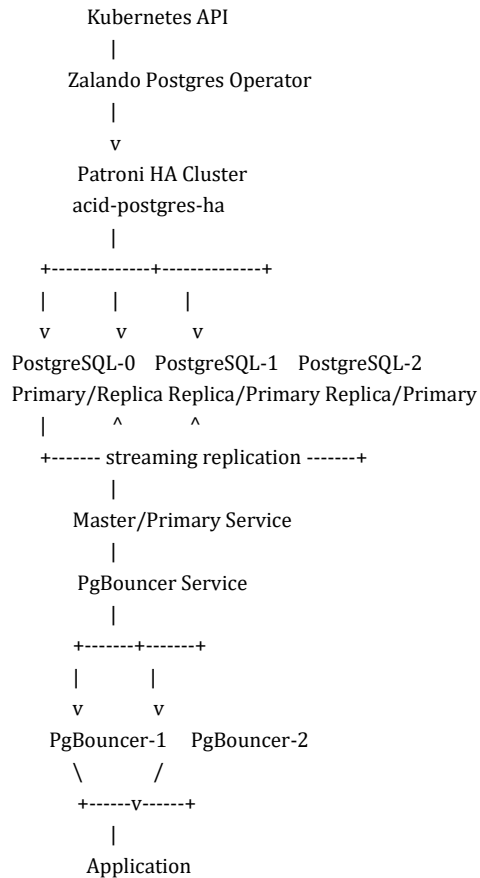
This corrected lab lets the Zalando Postgres Operator create and manage the StatefulSet, Patroni configuration, services, secrets, bootstrap, replica cloning, and PgBouncer integration.

Note: Do not reuse the PVCs or database data from the failed three-independent-primary deployment.

2. Learning Objectives

- Install the Zalando Postgres Operator on Kubernetes.
- Create one PostgreSQL 18 cluster containing three Patroni-managed instances.
- Verify exactly one primary and two replicas.
- Verify that all members belong to the same PostgreSQL database system.
- Enable an operator-managed PgBouncer connection pooler with two instances.
- Connect a banking application/client through the PgBouncer service.
- Create test data and verify read/write access.
- Perform a controlled Patroni switchover and verify service continuity.
- Understand the difference between Patroni HA, Kubernetes Services, and PgBouncer pooling.

3. Target Architecture



Patroni determines the writable primary. The operator maintains Kubernetes resources around the cluster. PgBouncer pools application connections; it does not perform Patroni leader election.

4. Prerequisites

- A working Kubernetes cluster.
- `kubectl` configured for the target cluster.
- Git installed on the Windows workstation.
- Internet/container-registry access from the cluster.
- A default Kubernetes StorageClass capable of provisioning ReadWriteOnce volumes.
- Permission to create cluster-scoped operator resources and namespace resources.
- At least enough cluster resources for three PostgreSQL pods plus the operator and two PgBouncer pods.

5. Lab 0 - Remove the Incorrect Previous Deployment

Because the previous lab produced three independent PostgreSQL primaries, remove that fresh lab deployment before starting the corrected exercise.

```
kubectl delete namespace postgres-ha
kubectl get namespace postgres-ha
```

Expected result: After deletion completes, `postgres-ha` is no longer listed.

Check persistent volumes:

```
kubectl get pv
```

Note: If old lab PVs remain because of the StorageClass reclaim policy, do not intentionally bind the corrected cluster to those old data volumes.

6. Lab 1 - Verify Kubernetes and Storage

```
kubectl version --client
kubectl cluster-info
kubectl get nodes -o wide
kubectl get storageclass
```

Expected result: The Kubernetes API is reachable, nodes are Ready, and at least one suitable StorageClass is available.

7. Lab 2 - Download the Postgres Operator

From Windows Command Prompt or PowerShell, change to your working directory and clone the operator repository at the release used for this lab:

```
cd /d E:\Docker\postgresdeployment
git clone --branch v2.0.2 --depth 1 https://github.com/zalando/postgres-operator.git
cd postgres-operator
```

Verify the repository:

```
git describe --tags --always
dir manifests
```

Expected result: The repository is available locally and the manifests directory is present.

8. Lab 3 - Install the Postgres Operator

The official operator quickstart supports deploying the supplied manifests with Kustomize. From the postgres-operator repository:

```
kubectl apply -k manifests
```

Watch the operator:

```
kubectl get pods -l name=postgres-operator -w
```

Press Ctrl+C when the operator is Running, then verify:

```
kubectl get pods -l name=postgres-operator
kubectl get deployment postgres-operator
kubectl get crd | findstr /l postgresql
```

Expected result: The postgres-operator pod is Running and the PostgreSQL custom resource definition is registered.

If the operator does not become Running:

```
kubectl get pods -l name=postgres-operator
kubectl describe deployment postgres-operator
kubectl logs deployment/postgres-operator --tail=200
```

9. Lab 4 - Create the PostgreSQL Namespace

```
kubectl create namespace postgres-ha
kubectl get namespace postgres-ha
```

Expected result: postgres-ha is Active.

10. Lab 5 - Create the 3-Instance PostgreSQL 18 Cluster with PgBouncer

Create a file named:

```
notepad E:\Docker\postgresdeployment\patroni\postgres-ha-operator.yaml
```

Paste:

```
apiVersion: "acid.zalan.do/v1"
kind: postgresql
metadata:
  name: acid-postgres-ha
  namespace: postgres-ha
spec:
  teamId: "acid"

  volume:
    size: 5Gi

  numberOfInstances: 3

  users:
    bank_app: []

  databases:
    banking: bank_app

  postgresql:
    version: "18"

  enableConnectionPooler: true

  connectionPooler:
    numberOfInstances: 2
    mode: "transaction"
    schema: "pooler"
    user: "pooler"
    resources:
      requests:
        cpu: 100m
        memory: 100Mi
      limits:
        cpu: "1"
        memory: 100Mi
```

Apply it:

```
cd /d E:\Docker\postgresdeployment\patroni
kubectl apply -f postgres-ha-operator.yaml
```

Expected result: Kubernetes accepts the postgresql custom resource.

11. Lab 6 - Watch the Operator Build the Cluster

```
kubectl get postgresql -n postgres-ha
kubectl get pods -n postgres-ha -w
```

The operator creates the StatefulSet and initializes one PostgreSQL member first. Remaining members are then created/cloned as replicas. PgBouncer pods are also created.

After the pods stabilize:

```
kubectl get pods -n postgres-ha -o wide
kubectl get pods -n postgres-ha -l application=spilo -L spilo-role
kubectl get svc -n postgres-ha
kubectl get pvc -n postgres-ha
```

Expected result: Three Spilo/PostgreSQL pods are Running, one is the master/primary role, two are replicas, and PgBouncer pooler pods are Running.

12. Lab 7 - Identify the Actual Pod and Service Names

Operator-created names differ from the earlier hand-written StatefulSet. Record them instead of assuming postgres-0/postgres-1/postgres-2.

```
kubectl get pods -n postgres-ha
kubectl get svc -n postgres-ha
kubectl get deployment -n postgres-ha
```

Typical resources include the cluster service, replica service, and a pooler service named similar to:

```
acid-postgres-ha
acid-postgres-ha-repl
acid-postgres-ha-pooler
```

Note: Use the names shown by your own kubectl output if they differ.

13. Lab 8 - Verify Patroni Roles

First list the database pods with their role label:

```
kubectl get pods -n postgres-ha -l application=spilo -L spilo-role
```

Expected result: Exactly one database pod is the primary/master and the other two are replicas.

Get one database pod name in Windows CMD:

```
kubectl get pods -n postgres-ha -l application=spilo
```

Then run patronictl inside a database pod, replacing POD_NAME:

```
kubectl exec -it -n postgres-ha POD_NAME -- patronictl list
```

If patronictl needs the generated configuration path in the image, inspect its help and process/configuration rather than supplying an invented scope option:

```
kubectl exec -it -n postgres-ha POD_NAME -- patronictl --help
kubectl exec -n postgres-ha POD_NAME -- curl -s http://localhost:8008/
```

Expected result: Patroni reports one leader/primary and two replicas in one cluster.

```

Administrator: Command Prompt
E:\Postgres\patronideployment>kubectl get pods -n postgres-ha -l application=spilo
NAME          READY   STATUS    RESTARTS   AGE
acid-postgres-ha-0  1/1     Running   0          4m23s
acid-postgres-ha-1  1/1     Running   0          2m44s
acid-postgres-ha-2  1/1     Running   0          2m40s

E:\Postgres\patronideployment>kubectl exec -it -n postgres-ha acid-postgres-ha-0 -- patronictl list
+ Cluster: acid-postgres-ha (7688892737475239995) -----+
| Member      | Host      | Role   | State   | TL | Receive LSN | Lag | Replay LSN | Lag |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| acid-postgres-ha-0 | 10.1.0.16 | Leader | running | 1  | 0/5252488   | 0   | 0/5252488   | 0   |
| acid-postgres-ha-1 | 10.1.0.17 | Replica | streaming | 1  | 0/5252488   | 0   | 0/5252488   | 0   |
| acid-postgres-ha-2 | 10.1.0.18 | Replica | streaming | 1  | 0/5252488   | 0   | 0/5252488   | 0   |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

E:\Postgres\patronideployment>

```

14. Lab 9 - Prove All Three Members Belong to One PostgreSQL Cluster

For each database pod, query the Patroni REST API:

```

kubectl exec -n postgres-ha POD_0 -- curl -s http://localhost:8008/
kubectl exec -n postgres-ha POD_1 -- curl -s http://localhost:8008/
kubectl exec -n postgres-ha POD_2 -- curl -s http://localhost:8008/

```

Compare the results.

Property	Correct HA cluster
Patroni scope	Same cluster scope on all members
Role	One primary; two replicas
State	Running/healthy
database_system_identifier	Same identifier on all three members
Patroni member name	Different member name per pod

This is the key validation that was missing from the failed raw StatefulSet lab.

15. Lab 10 - Inspect Operator-Created Credentials

The operator generates a Kubernetes Secret for manifest users. List the secrets:

```
kubectl get secrets -n postgres-ha
```

Look for a secret for bank_app whose name follows the operator credential-secret naming pattern. Inspect the secret metadata:

```
kubectl describe secret -n postgres-ha bank_app.acid-postgres-ha.credentials.postgresql.acid.zalan.do
```

To display the generated username and password using kubectl templates:

```
kubectl get secret -n postgres-ha bank_app.acid-postgres-ha.credentials.postgresql.acid.zalan.do -o jsonpath="{.data.username}"
kubectl get secret -n postgres-ha bank_app.acid-postgres-ha.credentials.postgresql.acid.zalan.do -o jsonpath="{.data.password}"
```

Note: Those values are base64 encoded. For a training environment, you can decode them with a local base64-capable tool or PowerShell. Avoid printing production credentials into shared terminals or screenshots.

16. Lab 11 - Verify PgBouncer

```
kubectl get deployment -n postgres-ha
kubectl get pods -n postgres-ha -l application=db-connection-pooler
kubectl get svc -n postgres-ha | findstr pooler
```

Expected result: The master connection pooler has two PgBouncer pods and a pooler Service.

Inspect pooler logs:

```
kubectl logs -n postgres-ha deployment/acid-postgres-ha-pooler --tail=100
```

Note: If your deployment name differs, substitute the name returned by `kubectl get deployment -n postgres-ha`.

17. Lab 12 - Connect Through PgBouncer

Create a temporary PostgreSQL client pod:

```
kubectl run pg-client --rm --it --restart=Never -n postgres-ha --image=postgres:18 -- bash
```

Inside the temporary client container, set the generated bank_app password and connect to the pooler:

```
export PGPASSWORD='GENERATED_BANK_APP_PASSWORD'
```

```
psql -h acid-postgres-ha-pooler -p 5432 -U bank_app -d banking
```

Inside psql:

```
SELECT current_database();
SELECT current_user;
SELECT pg_is_in_recovery();
SELECT inet_server_addr();
```

Expected result: The connection succeeds through PgBouncer and `pg_is_in_recovery()` is false because the master pooler targets the writable service.

18. Lab 13 - Banking Data Test

Inside the banking database:

```
CREATE TABLE accounts (
  account_id integer PRIMARY KEY,
  customer_name varchar(100) NOT NULL,
  balance numeric(12,2) NOT NULL
);
```

```
INSERT INTO accounts VALUES
(101, 'Customer A', 10000.00),
(102, 'Customer B', 15000.00),
(103, 'Customer C', 20000.00);
```

```
SELECT * FROM accounts ORDER BY account_id;
```

```
UPDATE accounts
SET balance = balance - 500
```

```
WHERE account_id = 101;
```

```
SELECT * FROM accounts WHERE account_id = 101;
```

Expected result: DDL and DML complete successfully through PgBouncer.

19. Lab 14 - Verify Streaming Replication

Find the primary pod:

```
kubecttl get pods -n postgres-ha -l application=spilo -L spilo-role
```

On the primary, query replication status. Replace PRIMARY_POD:

```
kubecttl exec -it -n postgres-ha PRIMARY_POD -- psql -U postgres -d postgres -c "SELECT application_name, client_addr, state, sync_state FROM pg_stat_replication;"
```

Expected result: Replica connections are visible and their state is streaming.

20. Lab 15 - Controlled Patroni Switchover

Record the current roles before changing anything:

```
kubecttl get pods -n postgres-ha -l application=spilo -L spilo-role
```

```
kubecttl exec -n postgres-ha PRIMARY_POD -- curl -s http://localhost:8008/
```

Open Patroni's CLI in one database pod:

```
kubecttl exec -it -n postgres-ha PRIMARY_POD -- patronictl
```

Use the patronictl help for the installed Patroni version and execute a controlled switchover interactively:

```
patronictl switchover --help
```

Note: The exact prompts/options can vary by Patroni version. Use the installed version's help rather than copying an option from another Patroni release.

After the switchover:

```
kubecttl get pods -n postgres-ha -l application=spilo -L spilo-role
```

```
kubecttl get svc -n postgres-ha
```

Expected result: A different member holds the primary/master role while the cluster remains healthy.

21. Lab 16 - Test PgBouncer After Switchover

Reconnect through the same PgBouncer service:

```
psql -h acid-postgres-ha-pooler -p 5432 -U bank_app -d banking
```

Then:

```
SELECT pg_is_in_recovery(), inet_server_addr();
```

```
UPDATE accounts
```

```
SET balance = balance + 100
```

```
WHERE account_id = 102;
```

```
SELECT * FROM accounts WHERE account_id = 102;
```

Expected result: The client can reconnect through the unchanged pooler endpoint and writes are routed to the current writable primary.

Note: Existing connections can be interrupted during failover. Real applications should implement appropriate reconnect/retry behavior.

22. Lab 17 - Pod Recovery Test

Choose a replica pod, not the current primary, and delete it:

```
kubectl delete pod -n postgres-ha REPLICA_POD
kubectl get pods -n postgres-ha -w
```

Expected result: Kubernetes/operator recreates the member and Patroni returns it to the cluster as a replica.

Verify again:

```
kubectl get pods -n postgres-ha -l application=spilo -L spilo-role
kubectl get pvc -n postgres-ha
```

23. Troubleshooting

Symptom	Commands	Interpretation / next area
Operator not Running	<code>kubectl logs deployment/postgres-operator --tail=200</code>	Check RBAC, image pull, CRD/operator configuration.
PostgreSQL CR not progressing	<code>kubectl describe postgresql acid-postgres-ha -n postgres-ha</code>	Check operator events and validation errors.
Database pod Pending	<code>kubectl describe pod POD -n postgres-ha</code> ; <code>kubectl get pvc -n postgres-ha</code>	Usually storage or scheduling.
Database pod CrashLoopBackOff	<code>kubectl logs POD -n postgres-ha --previous</code>	Inspect bootstrap/Patroni/PostgreSQL errors.
Three primaries again	<code>curl localhost:8008</code> on each DB pod	Stop. Verify they are operator-created members of the same CR; compare system identifiers.
No pooler pods	<code>kubectl get postgresql acid-postgres-ha -n postgres-ha -o yaml</code>	Confirm <code>enableConnectionPooler</code> and <code>connectionPooler</code> configuration.
Pooler authentication failure	<code>kubectl logs deployment/acid-postgres-ha-pooler -n postgres-ha</code>	Use operator-generated application credentials and verify database/user.
Write is read-only	<code>SELECT pg_is_in_recovery();</code> inspect target service	Client may be reaching a replica endpoint instead of the master pooler.

24. Production Considerations

- Pin and validate operator, Spilo/PostgreSQL, and PgBouncer versions before production rollout.
- Use appropriate StorageClasses, encryption, snapshots, backup/PITR, and tested restore procedures.
- Replication is not a substitute for backup.
- Apply resource requests/limits, topology spread or anti-affinity, and PodDisruptionBudgets suitable for the platform.
- Use TLS, NetworkPolicies, and secure secret-handling practices.
- Monitor Patroni role changes, replication lag, PostgreSQL health, PgBouncer saturation, storage and Kubernetes events.
- Test failover and application retry behavior under realistic load.

- Review operator upgrades and PostgreSQL major-version upgrades as separate controlled lifecycle activities.

25. Cleanup

Delete only the lab database cluster:

```
kubectl delete -f postgres-ha-operator.yaml
kubectl get pods -n postgres-ha
kubectl get pvc -n postgres-ha
```

Delete the lab namespace when no longer needed:

```
kubectl delete namespace postgres-ha
```

Remove the operator only if it is no longer needed by any PostgreSQL clusters:

```
cd /d E:\Docker\postgresdeployment\postgres-operator
kubectl delete -k manifests
```

Note: Do not remove a shared operator while other operator-managed PostgreSQL clusters still depend on it.

26. Final Validation Checklist

- ☐ Postgres Operator is Running.
- ☐ acid-postgres-ha custom resource exists.
- ☐ Three PostgreSQL/Spilo pods are Running.
- ☐ Exactly one member is primary/master and two are replicas.
- ☐ All three Patroni REST responses represent the same cluster.
- ☐ All three members share the same database_system_identifier.
- ☐ Two PgBouncer pooler pods are Running.
- ☐ bank_app can connect to banking through acid-postgres-ha-pooler.
- ☐ CREATE/INSERT/UPDATE succeeds through PgBouncer.
- ☐ pg_stat_replication shows streaming replicas.
- ☐ A controlled switchover changes the primary.
- ☐ The PgBouncer service name remains unchanged after switchover.
- ☐ A deleted replica is recreated and rejoins the cluster.

27. Key Concepts Recap

Component	Responsibility
PostgreSQL	Stores and processes database data.
Patroni	Leader election, PostgreSQL HA orchestration and failover coordination.
Zalando Postgres Operator	Creates/manages Kubernetes resources and PostgreSQL cluster lifecycle.

Kubernetes Service	Provides stable network endpoints despite pod changes.
PgBouncer	Pools client connections and reduces PostgreSQL connection overhead.
PersistentVolume/PVC	Provides durable database storage beyond pod lifetime.

28. References Used to Correct This Lab

This manual was corrected against the current Zalando Postgres Operator documentation/repository. The operator quickstart documents Kustomize installation and the operator-created PostgreSQL resources; the user/manifest documentation describes numberOfInstances, PostgreSQL version, generated user secrets, and enableConnectionPooler/connectionPooler configuration. The v2.0.2 repository compatibility table lists PostgreSQL 14–18 support.

- [Zalando Postgres Operator - Quickstart](#)
- [Zalando Postgres Operator - User Guide](#)
- [Zalando Postgres Operator - Cluster Manifest Reference](#)
- [Zalando Postgres Operator - GitHub repository / release compatibility](#)